

Regression Models: A Concise Tutorial

Understanding how different machine learning models behave across various datasets requires examining their underlying mathematical foundations, assumptions, and hyperparameter influences. This analysis explores model characteristics using a sinusoidal dataset as an illustrative example, but the insights apply broadly to regression problems.

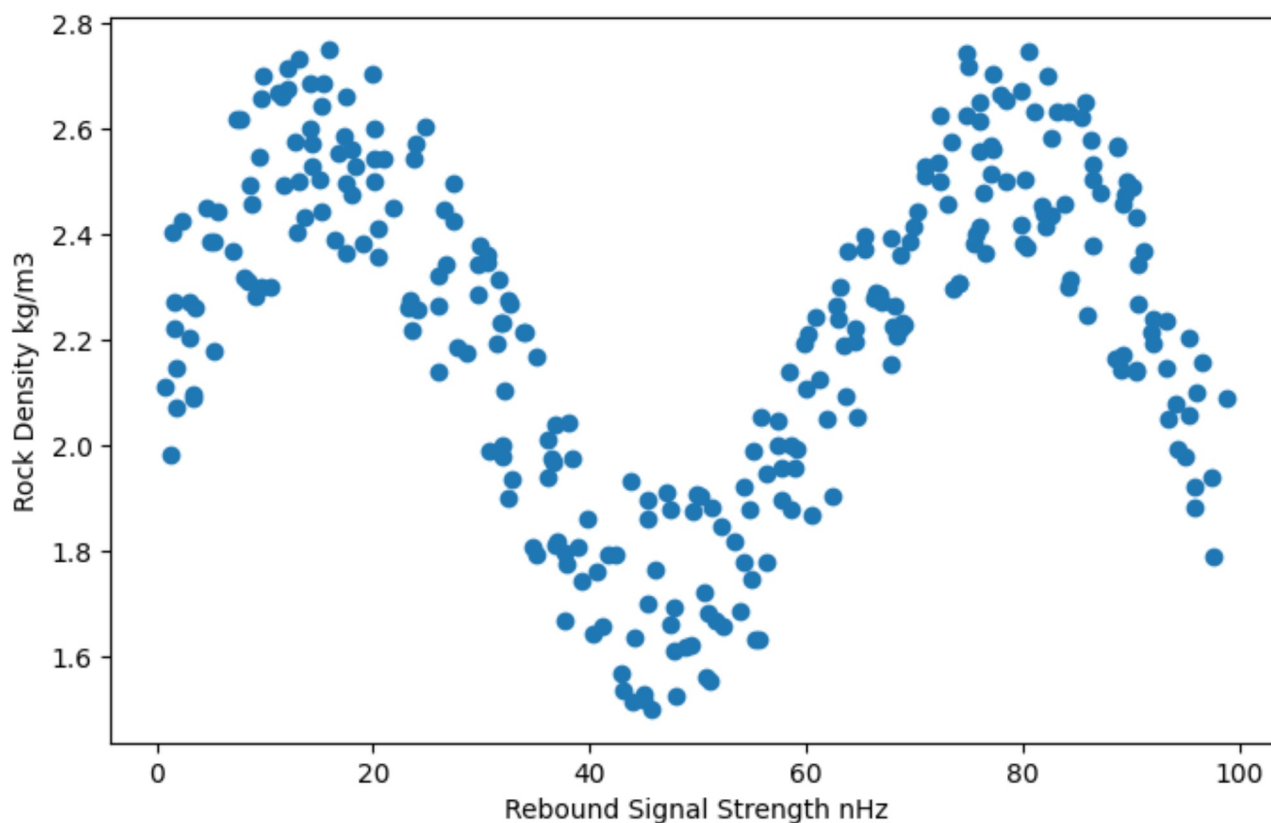
Data

```
In [2]: df = pd.read_csv('/kaggle/input/rock-density-xray/rock_density_xray.csv')
df.head()
```

```
Out[2]:
```

	Rebound Signal Strength nHz	Rock Density kg/m3
0	72.945124	2.456548
1	14.229877	2.601719
2	36.597334	1.967004
3	9.578899	2.300439
4	21.765897	2.452374

```
In [3]: plt.figure(figsize=(8, 5))
plt.scatter(df[df.columns[0]], df[df.columns[1]]);
plt.xlabel(df.columns[0]);
plt.ylabel(df.columns[1]);
```



```
In [7]: from sklearn.model_selection import train_test_split
```

```
In [8]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, random_state=42)
```

Linear Regression

Mathematical Foundation & Motivation

Developed by Carl Friedrich Gauss (~1800s), foundation of statistical modeling

- **Core Philosophy:** Assumes the simplest possible relationship (Occam's razor)
- **Mathematical Elegance:** Closed-form solution exists, computationally efficient
- **Statistical Foundation:** Based on least squares estimation with strong theoretical guarantees

Assumptions & Characteristics

- **Core Assumption:** Linear relationship between features and target: $y = X\beta + \varepsilon$
- **Distributional Assumptions:** Errors are normally distributed, independent, and homoscedastic
- **Mathematical Form:** $y = \beta_0 + \beta_1x_1 + \dots + \beta_px_p$
- **Flexibility:** Extremely rigid, no ability to capture non-linear patterns
- **Interpretability:** Perfect - coefficients have direct meaning

```
In [11]: from sklearn.linear_model import LinearRegression, Ridge, Lasso
```

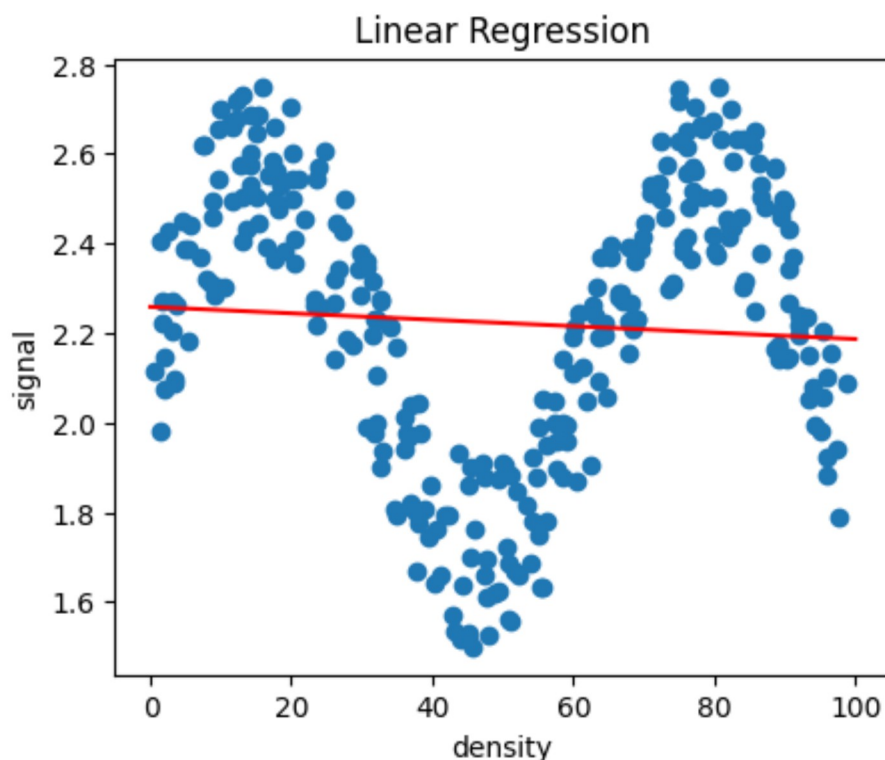
```
In [12]: model = LinearRegression()
```

```
In [13]: run_n_test(model, "Linear Regression")
```

Mean Absolute Error : 0.2630059641510617

Mean Squared Error : 0.1005845264173819

Root Mean Squared Error : 0.3171506367917017



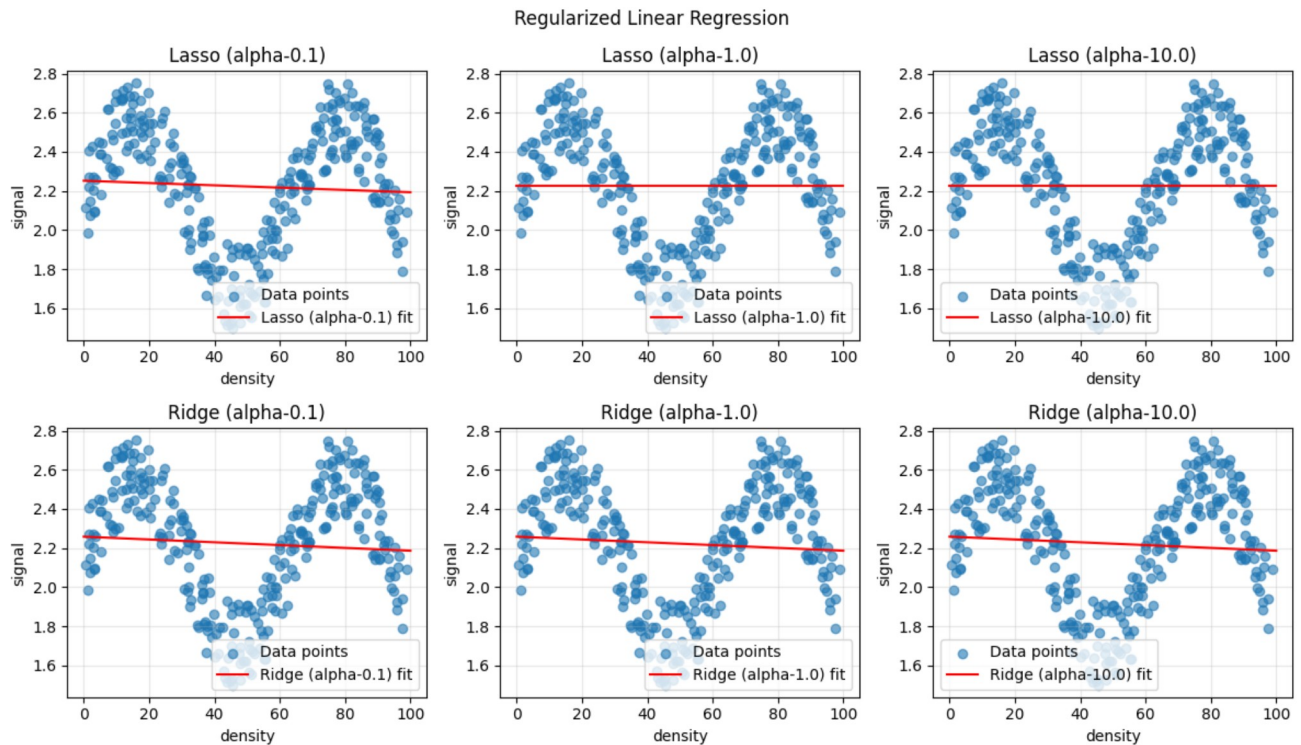
Key Hyperparameters

- **None in basic form** - this is both a strength and limitation
- **Regularized variants:**

- Ridge (L2): α controls shrinkage strength
- Lasso (L1): α controls feature selection strength
- Elastic Net: α (overall strength) and l1_ratio (L1 vs L2 balance)

```
In [15]: run_n_test_regu("Regularized Linear Regression", alphas=[0.1, 1.0, 10.0])
```

Lasso (alpha=0.1): MAE:0.2620, MSE:0.1001
 Lasso (alpha=1.0): MAE:0.2585, MSE:0.0981
 Lasso (alpha=10.0): MAE:0.2585, MSE:0.0981
 Ridge (alpha=0.1): MAE:0.2630, MSE:0.1006
 Ridge (alpha=1.0): MAE:0.2630, MSE:0.1006
 Ridge (alpha=10.0): MAE:0.2630, MSE:0.1006



Graph Behavior

- **Shape:** Straight line regardless of true relationship complexity
- **Performance:** High bias, low variance - consistently underfits complex relationships
- **Visual Result:** Diagonal line that represents the "best fit" linear approximation
- **Residual Patterns:** Systematic patterns in residuals indicate model inadequacy

When to Use

- **Ideal Scenarios:** True linear relationships, interpretability is paramount, baseline modeling
- **Adequate Scenarios:** Simple relationships, limited data, need for statistical inference
- **Avoid When:** Non-linear relationships, complex interactions, purely predictive tasks
- **Industry Applications:** Economics (simple models), medical research (interpretable coefficients)

Polynomial Regression

Extension of linear regression to capture curvature

- **Core Philosophy:** Any smooth function can be approximated by polynomials (Weierstrass

theorem)

- **Mathematical Elegance:** Remains linear in parameters, maintains interpretability
- **Practical Motivation:** Simple way to add non-linearity while keeping linear algebra framework

Assumptions & Characteristics

- **Core Assumption:** Relationship can be expressed as polynomial terms
- **Mathematical Form:** $y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$
- **Flexibility:** Increases dramatically with degree
- **Feature Engineering:** Creates interaction terms automatically

```
In [16]: from sklearn.preprocessing import PolynomialFeatures
```

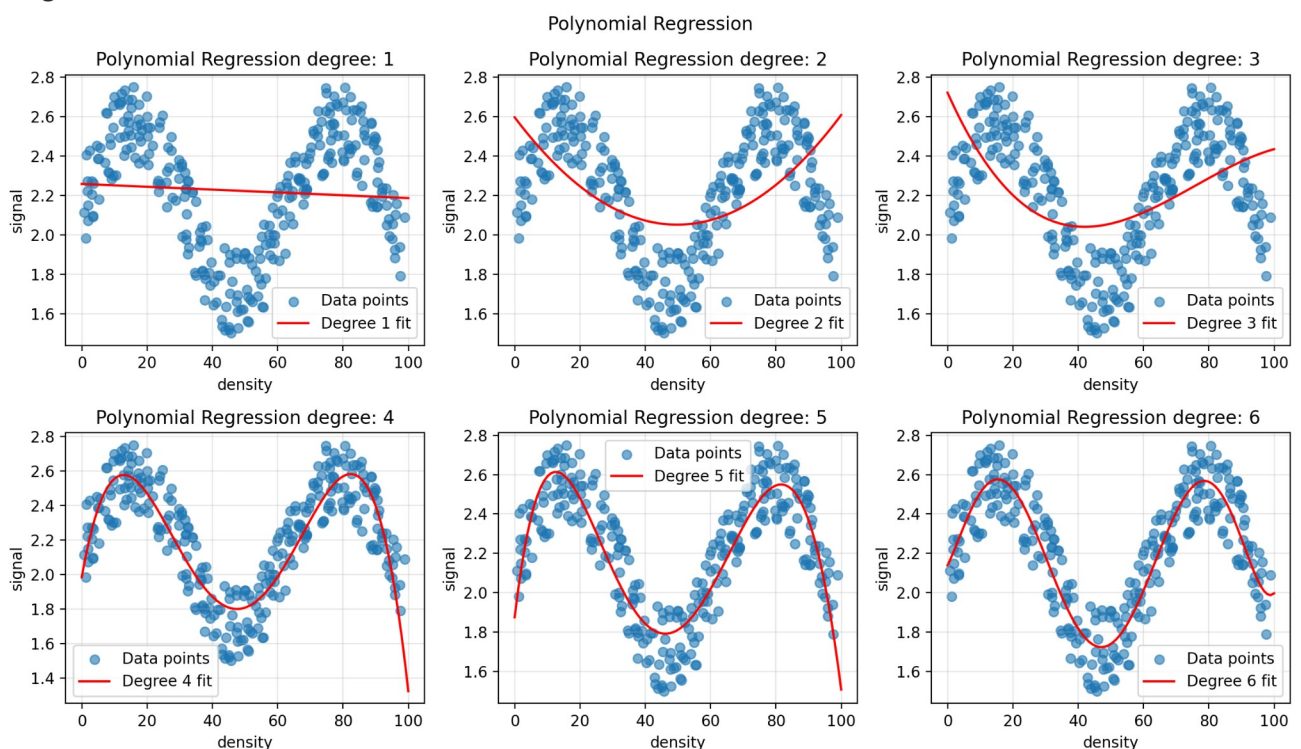
```
In [17]: poly_linreg = LinearRegression()
```

Key Hyperparameters

- **Degree (n):** Most critical parameter
 - Low degree (1-3): Underfitting risk, smooth curves
 - Medium degree (4-8): Often optimal balance
 - High degree (>10): Overfitting risk, wild oscillations
- **Include_bias:** Whether to include intercept term
- **Interaction_only:** For multivariate cases, whether to include cross-terms only

```
In [19]: run_n_test_subp(poly_linreg, "Polynomial Regression", [1,2,3,4,5,6])
```

Degree 1 :	MAE:0.2630	MSE:0.1006
Degree 2 :	MAE:0.2720	MSE:0.0997
Degree 3 :	MAE:0.2566	MSE:0.0902
Degree 4 :	MAE:0.1273	MSE:0.0231
Degree 5 :	MAE:0.1234	MSE:0.0215
Degree 6 :	MAE:0.1278	MSE:0.0205



Graph Behavior

- **Low Degree** (2-3): Smooth parabolic/cubic curves, may miss oscillations
- **Medium Degree** (4-6): Better approximation, captures multiple turning points
- **High Degree** (8+): Can fit complex patterns but prone to overfitting with extreme boundary behavior
- **Visual Result:** Always smooth and continuous, but may have unwanted oscillations

When to Use

- **Ideal Scenarios:** Known polynomial relationships, smooth non-linear patterns, small feature spaces
 - **Adequate Scenarios:** Exploratory analysis, baseline for non-linear modeling, interpretable non-linearity needed
 - **Avoid When:** High-dimensional data, non-smooth relationships, extrapolation beyond training range
 - **Industry Applications:** Engineering (physical laws), economics (utility curves), scientific modeling
-

K-Nearest Neighbors

One of the oldest ML algorithms, based on similarity principle

- **Core Philosophy:** "You are the average of your neighbors" - local similarity assumption
- **Mathematical Foundation:** Non-parametric method based on distance metrics
- **Practical Motivation:** Simple, intuitive, no assumptions about data distribution

Assumptions & Characteristics

- **Core Assumption:** Similar inputs produce similar outputs (smoothness assumption)
- **Mathematical Form:** $\hat{y} = (1/k) \sum y_i$ for k nearest neighbors
- **Flexibility:** High locally, can adapt to any function shape
- **Non-parametric:** Doesn't assume specific functional form

```
In [20]: from sklearn.neighbors import KNeighborsRegressor
```

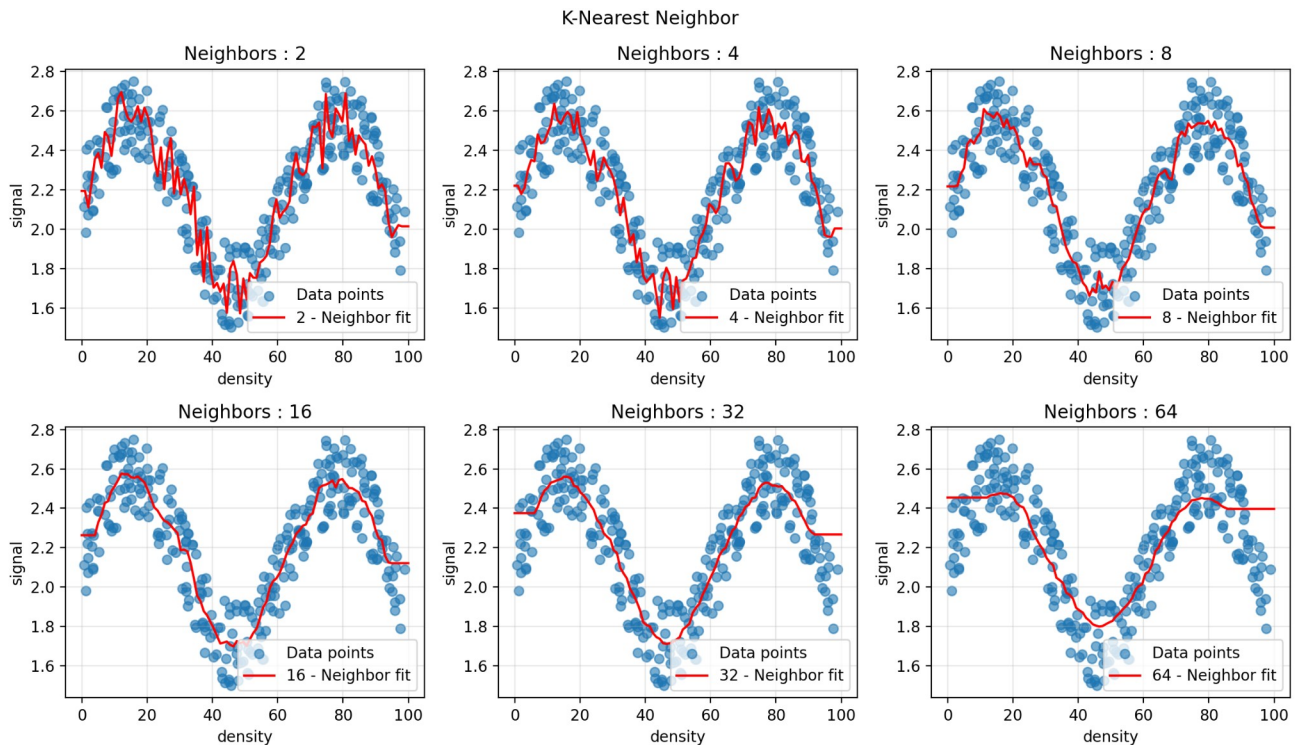
Key Hyperparameters

- **N_neighbors (k):** Number of neighbors to consider (most critical)
 - Small k (1-3): Low bias, high variance, captures local patterns
 - Medium k (5-15): Often optimal balance
 - Large k (20+): High bias, low variance, smoother predictions
- **Weights:** How to weight neighbors
 - Uniform: All neighbors weighted equally
 - Distance: Closer neighbors weighted more heavily
- **Metric:** Distance function
 - Euclidean: Standard choice for continuous features
 - Manhattan: Good for high dimensions
 - Minkowski: Generalizes both above

- **Algorithm:** Method for finding neighbors (ball_tree, kd_tree, brute force)

```
In [22]: run_n_test_knn(title= "K-Nearest Neighbor", neighbors=[2, 4, 8, 16, 32, 64])
```

```
Neighbors 2 :    MAE:0.1402    MSE:0.0259
Neighbors 4 :    MAE:0.1342    MSE:0.0242
Neighbors 8 :    MAE:0.1219    MSE:0.0199
Neighbors 16 :   MAE:0.1305    MSE:0.0223
Neighbors 32 :   MAE:0.1448    MSE:0.0292
Neighbors 64 :   MAE:0.1667    MSE:0.0432
```



Graph Behavior

- **Shape:** Piecewise smooth curve that can follow local patterns well
- **Visual Result:** Somewhat choppy but can capture local relationships effectively
- **Boundaries:** May have discontinuities, especially with small k
- **Noise Sensitivity:** Highly sensitive to noise and irrelevant features
- **Curse of Dimensionality:** Performance degrades with high dimensions

When to Use

- **Ideal Scenarios:** Low-dimensional data, local patterns important, irregular decision boundaries
- **Adequate Scenarios:** Non-linear relationships, sufficient data density, baseline modeling
- **Avoid When:** High-dimensional data, sparse data, computational efficiency critical, many irrelevant features
- **Industry Applications:** Recommendation systems, image recognition, anomaly detection, imputation

Support Vector Machines

Developed by Vladimir Vapnik (1990s), based on statistical learning theory

- **Core Philosophy:** Maximize margin while tolerating errors within ϵ -tube

- **Mathematical Foundation:** Convex optimization with kernel trick for non-linearity
- **Practical Motivation:** Strong theoretical guarantees, kernel methods enable infinite-dimensional feature spaces

Assumptions & Characteristics

- **Core Assumption:** Depends on kernel choice - can range from linear to highly complex
- **Mathematical Form:** $f(x) = \sum(\alpha_i - \alpha_i^*)K(x_i, x) + b$
- **Flexibility:** Extremely high with appropriate kernel selection
- **Robustness:** ϵ -insensitive loss makes it robust to outliers

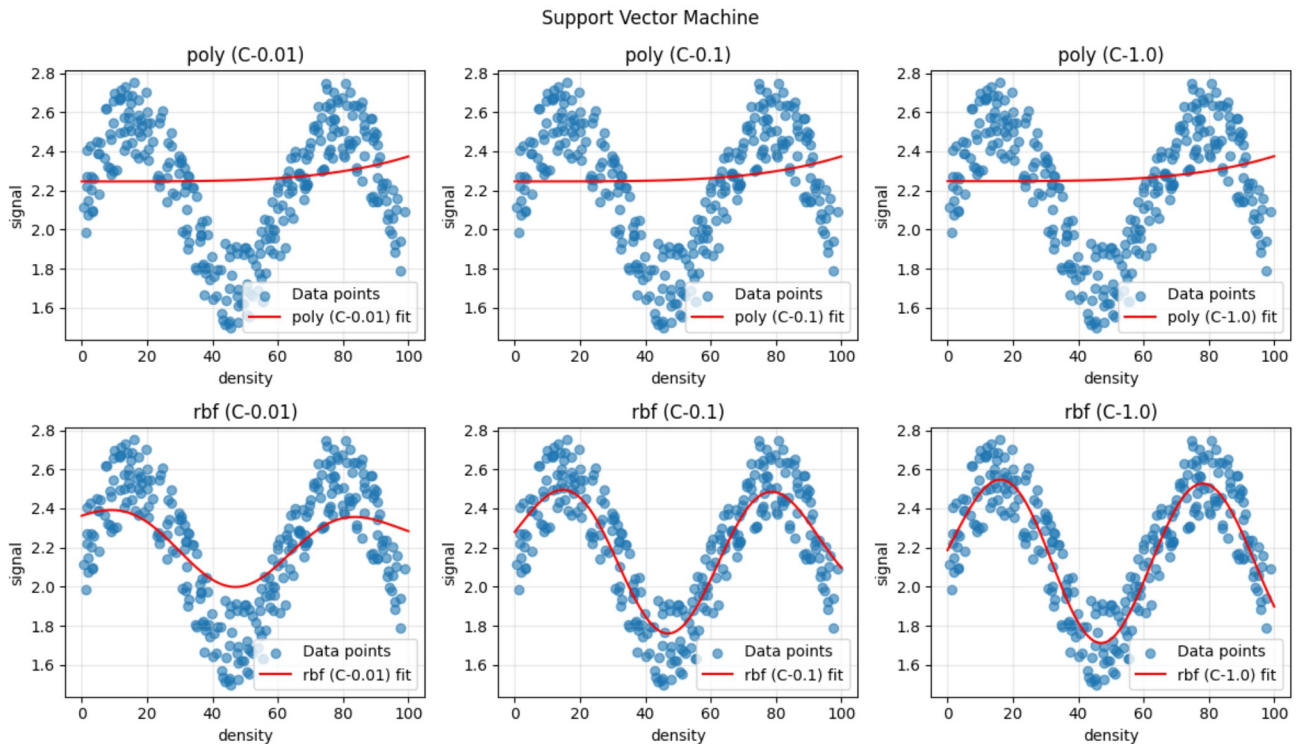
```
In [23]: from sklearn.svm import SVR
```

Key Hyperparameters

- **Kernel:** Choice of kernel function (most critical)
 - Linear: $K(x, x') = x^T x'$ (for linear relationships)
 - RBF/Gaussian: $K(x, x') = \exp(-\gamma x \cdot x')$ (most popular, handles non-linearity)
 - Polynomial: $K(x, x') = (\gamma x^T x' + r)^d$ (specific polynomial relationships)
- **C:** Regularization parameter (0.1-1000)
 - Higher C: Less tolerance for errors, may overfit
 - Lower C: More tolerance, may underfit
- **Epsilon (ϵ):** Width of ϵ -insensitive zone (0.01-1.0)
- **Gamma (γ):** RBF kernel parameter (0.001-10)
 - Higher γ : More complex decision boundary, may overfit
 - Lower γ : Smoother decision boundary

```
In [25]: run_n_test_svm("Support Vector Machine", kernel=["poly", "rbf"], C = [0.01, 0.1, 1.0])
```

```
poly (C=0.01): MAE:0.2518, MSE:0.0981
poly (C=0.1): MAE:0.2518, MSE:0.0981
poly (C=1.0): MAE:0.2519, MSE:0.0982
rbf (C=0.01): MAE:0.1983, MSE:0.0559
rbf (C=0.1): MAE:0.1342, MSE:0.0253
rbf (C=1.0): MAE:0.1280, MSE:0.0206
```



Graph Behavior

- **Linear Kernel:** Straight line (equivalent to linear regression with regularization)
- **RBF Kernel:** Smooth continuous curve that can follow complex patterns excellently
- **Polynomial Kernel:** Similar to polynomial regression but more robust to outliers
- **Visual Result:** Generally very smooth, excellent for continuous relationships

When to Use

- **Ideal Scenarios:** Non-linear relationships, outlier robustness needed, small-to-medium datasets
- **Adequate Scenarios:** High-dimensional data, when smooth predictions desired
- **Avoid When:** Very large datasets (computational complexity), need probabilistic outputs
- **Industry Applications:** Time series forecasting, signal processing, chemical/physical modeling

Decision Trees

Developed from decision theory and information theory (1960s-1980s)

- **Core Philosophy:** Hierarchical decision-making mimics human reasoning
- **Mathematical Foundation:** Based on information gain, entropy, or variance reduction
- **Practical Motivation:** Naturally handles non-linearity, interactions, and mixed data types

Assumptions & Characteristics

- **Core Assumption:** Data can be partitioned using recursive binary splits
- **Mathematical Form:** Piecewise constant functions over rectangular regions
- **Flexibility:** High capacity but creates discontinuous predictions
- **Interpretability:** Excellent - provides explicit decision rules

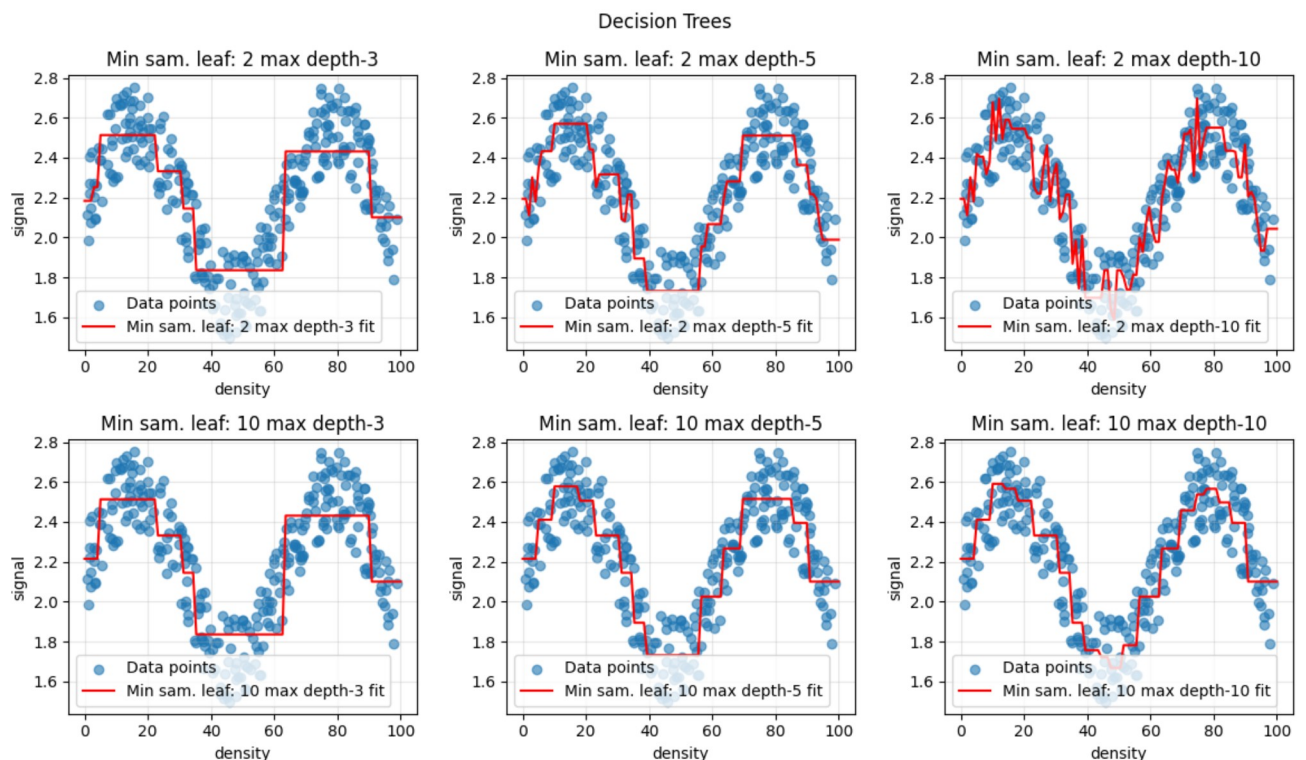
In [26]: `from sklearn.tree import DecisionTreeRegressor`

Key Hyperparameters

- **Max_depth:** Controls tree depth (most important for overfitting)
 - Shallow (2-5): High bias, low variance
 - Medium (6-10): Often optimal
 - Deep (>15): Overfitting risk
- **Min_samples_split:** Minimum samples required to split (controls granularity)
- **Min_samples_leaf:** Minimum samples in leaf nodes (smoothing effect)
- **Max_features:** Number of features considered at each split
- **Criterion:** Split quality measure (MSE for regression, Gini/entropy for classification)

In [28]: `run_n_test_dtree("Decision Trees", min_leaf=[2, 10], max_depths=[3, 5, 10])`

Min sam. leaf: 2 max depth-3: MAE:0.1569, MSE:0.0359
 Min sam. leaf: 2 max depth-5: MAE:0.1305, MSE:0.0230
 Min sam. leaf: 2 max depth-10: MAE:0.1353, MSE:0.0272
 Min sam. leaf: 10 max depth-3: MAE:0.1568, MSE:0.0358
 Min sam. leaf: 10 max depth-5: MAE:0.1259, MSE:0.0230
 Min sam. leaf: 10 max depth-10: MAE:0.1268, MSE:0.0231



Expected Graph Behavior

- **Shape:** Horizontal step functions creating "staircase" approximation
- **Visual Result:** Jagged, discontinuous curve following the general trend
- **Smoothness:** No smoothness - abrupt jumps between predicted values
- **Boundaries:** Sharp transitions at split points
- **Overfitting Pattern:** Very detailed steps that don't generalize

When to Use

- **Ideal Scenarios:** Categorical features, interpretability crucial, non-linear interactions, mixed data types
 - **Adequate Scenarios:** Exploratory analysis, feature importance assessment, baseline modeling
 - **Avoid When:** Smooth continuous relationships, small datasets, high noise, extrapolation needed
 - **Industry Applications:** Medical diagnosis, credit scoring, rule-based systems, feature engineering
-

Random Forest

Developed by Leo Breiman (2001), based on bagging and random subsampling

- **Core Philosophy:** "Wisdom of crowds" - ensemble methods reduce individual model weaknesses
- **Mathematical Foundation:** Bootstrap aggregation with random feature selection
- **Practical Motivation:** Addresses decision tree overfitting while maintaining interpretability

Assumptions & Characteristics

- **Assumption:** Ensemble of diverse trees can smooth out individual limitations
- **Mathematical Form:** Average of multiple step functions: $\hat{y} = (1/B)\sum_b f_b(x)$
- **Flexibility:** High, with built-in regularization through averaging and randomization
- **Robustness:** Handles missing values, mixed data types, and outliers well#### Assumptions & Characteristics

```
In [29]: from sklearn.ensemble import RandomForestRegressor
```

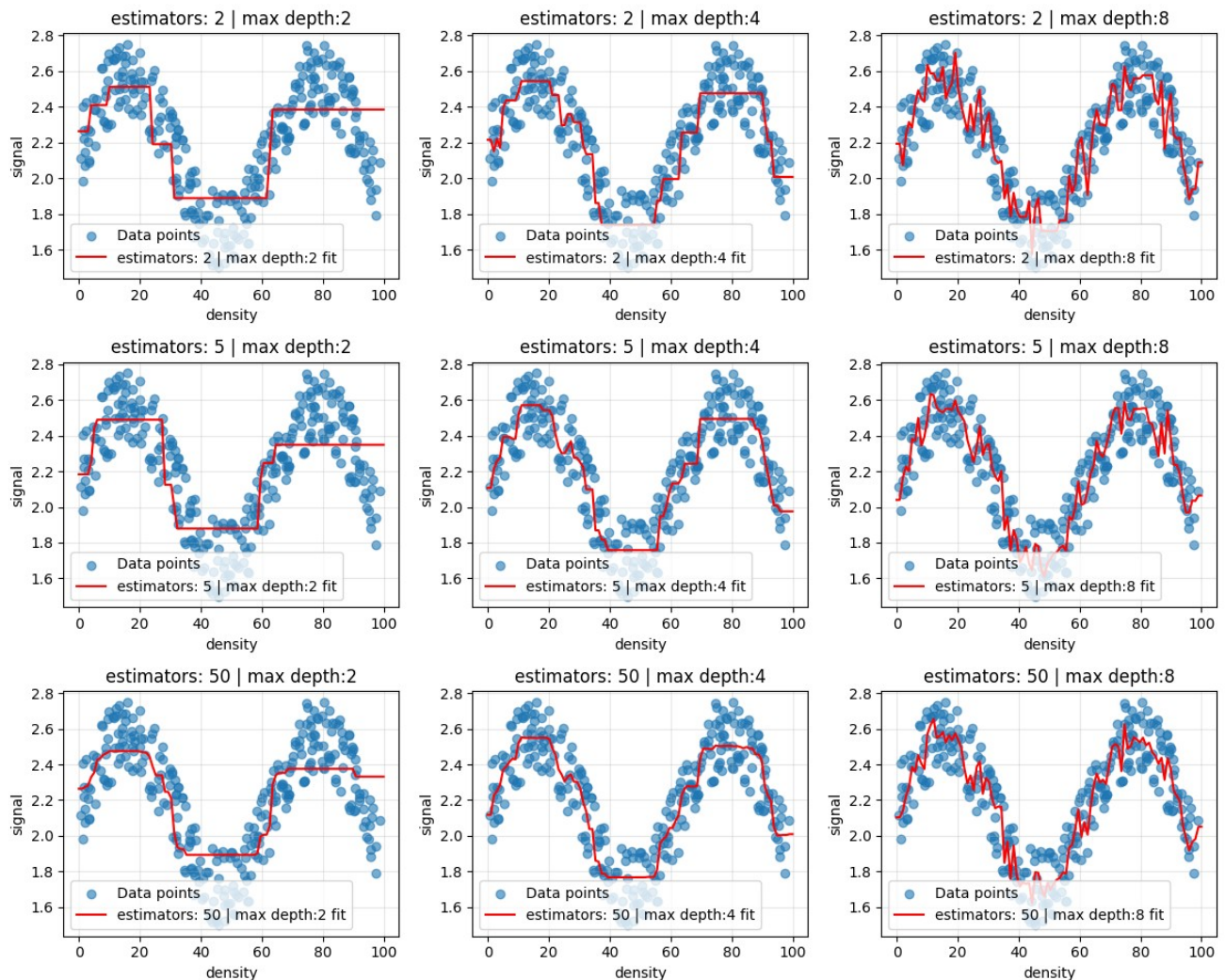
Key Hyperparameters

- **N_estimators:** Number of trees (more is generally better, diminishing returns after ~100-500)
- **Max_depth:** Depth of individual trees (often left unlimited for RF)
- **Min_samples_split/leaf:** Controls individual tree complexity
- **Max_features:** Features considered at each split
 - $\sqrt{p}$: Default for classification
 - $p/3$: Default for regression
 - Lower values increase diversity but may underfit
- **Bootstrap:** Whether to use bootstrap sampling (usually True)
- **Random_state:** For reproducibility

```
In [31]: run_n_test_rf("Random Forest", n_estim=[2, 5, 50], max_depths=[2, 4, 8])
```

estimators: 2 | max depth:2: MAE:0.1775, MSE:0.0474
 estimators: 2 | max depth:4: MAE:0.1210, MSE:0.0231
 estimators: 2 | max depth:8: MAE:0.1335, MSE:0.0286
 estimators: 5 | max depth:2: MAE:0.1623, MSE:0.0416
 estimators: 5 | max depth:4: MAE:0.1284, MSE:0.0229
 estimators: 5 | max depth:8: MAE:0.1314, MSE:0.0248
 estimators: 50 | max depth:2: MAE:0.1638, MSE:0.0405
 estimators: 50 | max depth:4: MAE:0.1220, MSE:0.0212
 estimators: 50 | max depth:8: MAE:0.1267, MSE:0.0234

Random Forest



Expected Graph Behavior

- **Shape:** Smoother than individual trees but still somewhat stepwise
- **Visual Result:** Less jagged than single tree, better approximation of curves than decision trees
- **Variance:** Reduced compared to single decision tree
- **Boundaries:** Softer transitions, but still some discontinuities
- **Comparison** to Boosting: Generally less smooth than gradient boosting methods

When to Use

- **Ideal Scenarios:** Mixed data types, feature importance needed, robust baseline, high-dimensional data
- **Adequate Scenarios:** Non-linear relationships, interpretability somewhat important, limited tuning time
- **Avoid When:** Very smooth relationships, small datasets, computational efficiency critical
- **Industry Applications:** Bioinformatics, image processing, general-purpose ML, feature selection

Boosted Trees

Boosting is a machine learning technique that combines multiple weak learners (models that perform slightly better than random guessing) to create a strong, highly accurate model. It focuses on iteratively improving the model by focusing on misclassified data points from previous iterations. This process reduces bias and often variance, leading to better predictive performance

Adaptive Boosting

Freund & Schapire (1997) - first practical boosting algorithm with strong theoretical backing

- **Core Philosophy:** "Focus on what you got wrong" - exponentially increase weights of misclassified examples
- **Mathematical Foundation:** Minimizes exponential loss through coordinate descent in function space
- **Practical Motivation:** Provides theoretical guarantee that training error decreases exponentially

Assumptions & Characteristics

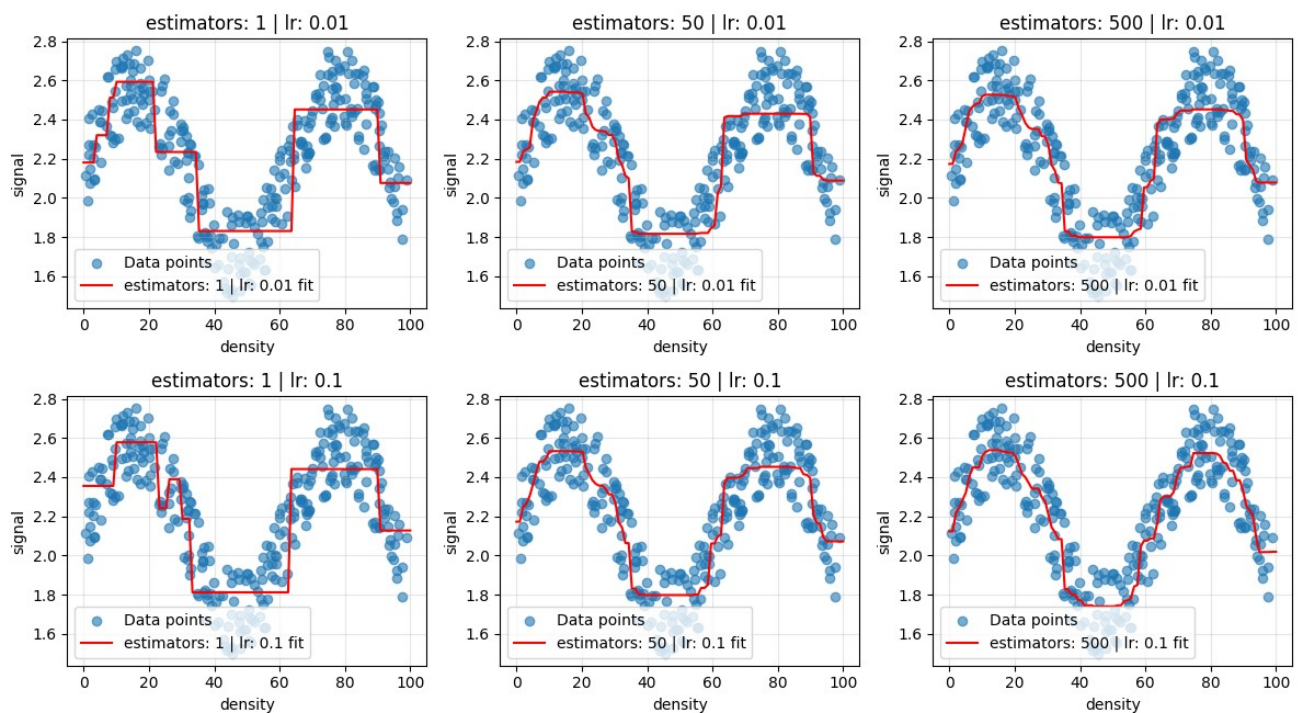
- **Core Assumption:** Each weak learner achieves error rate < 0.5 (better than random)
- **Mathematical Form:** Final hypothesis $H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$ where $\alpha_t = \frac{1}{2} \ln((1 - \epsilon_t)/\epsilon_t)$
- **Flexibility:** Limited by base learner choice, but excellent at combining simple rules
- **Weight Adaptation:** Exponential reweighting scheme: $w_i^{(t+1)} = w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))$

```
In [32]: from sklearn.ensemble import AdaBoostRegressor
```

```
In [34]: run_n_test_aboost("Adaptive Boosted Trees", n_estim=[1, 50, 500], lr = [0.01, 0.1])
```

```
estimators: 1 | lr: 0.01: MAE:0.1805, MSE:0.0467
estimators: 50 | lr: 0.01: MAE:0.1526, MSE:0.0313
estimators: 500 | lr: 0.01: MAE:0.1430, MSE:0.0276
estimators: 1 | lr: 0.1: MAE:0.1778, MSE:0.0436
estimators: 50 | lr: 0.1: MAE:0.1388, MSE:0.0262
estimators: 500 | lr: 0.1: MAE:0.1313, MSE:0.0226
```


Adaptive Boosted Trees



Key Hyperparameters

- **N_estimators:** Number of weak learners (50-500 typically)

AdaBoost can overfit with too many estimators unlike some theory suggests

- **Base_estimator:** Usually decision stumps (`max_depth=1`) or shallow trees

Learning_rate: Shrinks contribution of each classifier (0.1-2.0) Lower rates require more estimators but often generalize better

- **Algorithm:** SAMME (discrete) vs SAMME.R (real-valued, faster convergence)

Expected Graph Behavior

- **Shape:** With decision stumps: piecewise constant with sharp transitions
- **Visual Result:** Complex decision boundaries built from simple axis-aligned cuts
- **Granularity:** Relatively coarse individual decisions but refined through weighting
- **Adaptability:** Creates intricate boundaries around difficult examples
- **Characteristic Pattern:** Often shows "staircase" effects due to discrete weak learners

When to Use

- **Ideal Scenarios:** Binary classification with simple features, interpretability desired, theoretical guarantees important
- **Adequate Scenarios:** Multi-class problems, text classification, when data is not too noisy
- **Avoid When:** Very noisy datasets (sensitive to outliers), regression tasks, high-dimensional sparse data
- **Industry Applications:** Face detection (Haar cascades), text mining, bioinformatics, computer vision preprocessing

Gradient Boosting

- **Core Philosophy:** Sequential learning from mistakes - each model corrects previous errors
- **Mathematical Foundation:** Functional gradient descent in function space
- **Practical Motivation:** Achieves state-of-the-art performance on tabular data

Assumptions & Characteristics

- **Core Assumption:** Complex functions can be approximated by combining many weak learners
- **Mathematical Form:** Additive model: $F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$
- **Flexibility:** Extremely high, with sophisticated regularization mechanisms
- **Sequential Learning:** Each iteration specifically targets current model's weaknesses

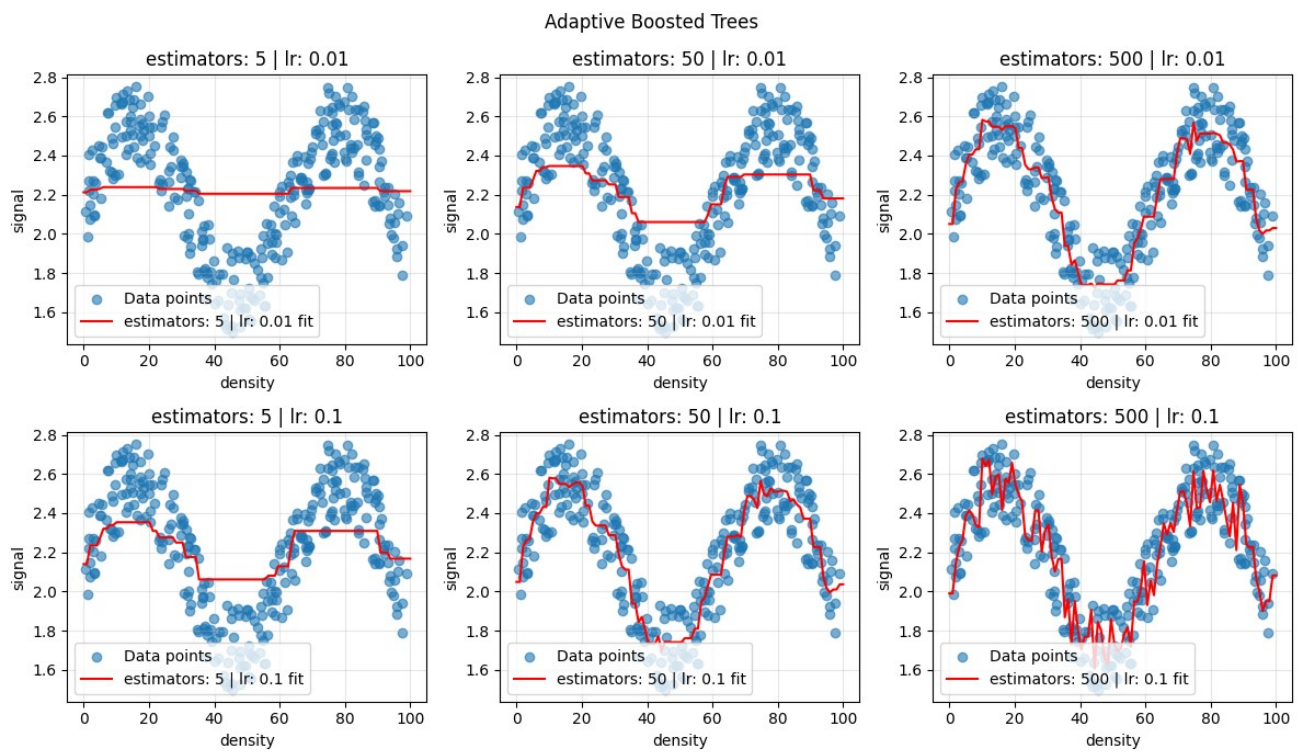
```
In [35]: from sklearn.ensemble import GradientBoostingRegressor
```

Key Hyperparameters

- **Learning_rate (η):** Step size shrinkage (0.01-0.3)
 - Lower values need more estimators but often generalize better
- **N_estimators:** Number of boosting rounds (50-10000+)
- **Max_depth:** Tree depth (3-10, deeper than RF typically)
- **Subsample:** Fraction of samples used per tree (0.5-1.0)
- **Colsample_bytree:** Fraction of features used per tree
- **Reg_alpha/lambda:** L1/L2 regularization terms
- **Min_child_weight:** Minimum sum of instance weight in child
- **Gamma:** Minimum loss reduction for split (XGBoost)

```
In [37]: run_n_test_aboost("Adaptive Boosted Trees", n_estim=[5, 50, 500], lr = [0.01, 0.1])
```

```
estimators: 5 | lr: 0.01:  MAE:0.2499, MSE:0.0921
estimators: 50 | lr: 0.01:  MAE:0.1958, MSE:0.0571
estimators: 500 | lr: 0.01:  MAE:0.1289, MSE:0.0217
estimators: 5 | lr: 0.1:  MAE:0.1959, MSE:0.0560
estimators: 50 | lr: 0.1:  MAE:0.1290, MSE:0.0218
estimators: 500 | lr: 0.1:  MAE:0.1175, MSE:0.0235
```



Expected Graph Behavior

- **Shape:** Much smoother than individual trees due to gradient-based optimization
- **Visual Result:** Can approximate complex patterns reasonably well, smoother than random forest
- **Granularity:** Still somewhat step-like but with much finer steps than single trees
- **Adaptability:** Excellent at capturing local patterns and complex relationships
- **Overfitting Control:** Sophisticated regularization prevents overfitting better than RF

When to Use

- **Ideal Scenarios:** Tabular data competitions, complex non-linear relationships, feature interactions important
- **Adequate Scenarios:** Most supervised learning problems, when high accuracy is needed
- **Avoid When:** Very small datasets, simple linear relationships, interpretability is crucial, real-time inference needed
- **Industry Applications:** Finance (risk modeling), marketing (customer analytics), healthcare (outcome prediction)

Gaussian Process Regression

Rooted in spatial statistics and Bayesian inference (1960s-1990s)

- **Core Philosophy:** Bayesian approach to function learning - distributions over functions
- **Mathematical Foundation:** Multivariate Gaussian distributions, kernel methods, Bayesian inference
- **Practical Motivation:** Provides uncertainty quantification and principled handling of noise

Assumptions & Characteristics

- **Core Assumption:** Function values are drawn from multivariate Gaussian distribution
- **Mathematical Form:** $f(x) \sim \text{GP}(\mu(x), k(x, x'))$ where k is the kernel function
- **Flexibility:** Extremely high, determined by kernel choice
- **Uncertainty:** Provides prediction intervals, not just point estimates

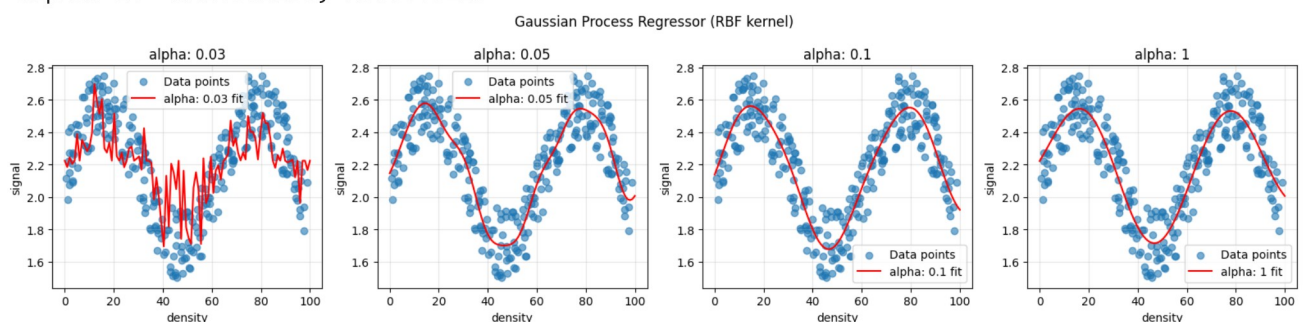
```
In [38]: from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, Matern, RationalQuadratic, WhiteKernel
```

Key Hyperparameters

- **Kernel:** Choice of covariance function (most critical)
 - RBF: $k(x, x') = \sigma^2 \exp(-||x-x'||^2/(2l^2))$ (most common, smooth functions)
 - Matérn: More general, controls smoothness explicitly
 - Periodic: $k(x, x') = \exp(-2\sin^2(\pi|x-x'|/p)/l^2)$ (perfect for periodic data!)
 - Linear: For linear trends
 - Combination: Sum/product of kernels
- **Length_scale (l):** Controls input correlation distance
 - Small l: Rapid variation, more wiggly functions
 - Large l: Smooth variation, simpler functions
- **Signal_variance (σ^2):** Overall function variation
- **Noise_level:** Observation noise assumption
- **Alpha:** Nugget effect for numerical stability

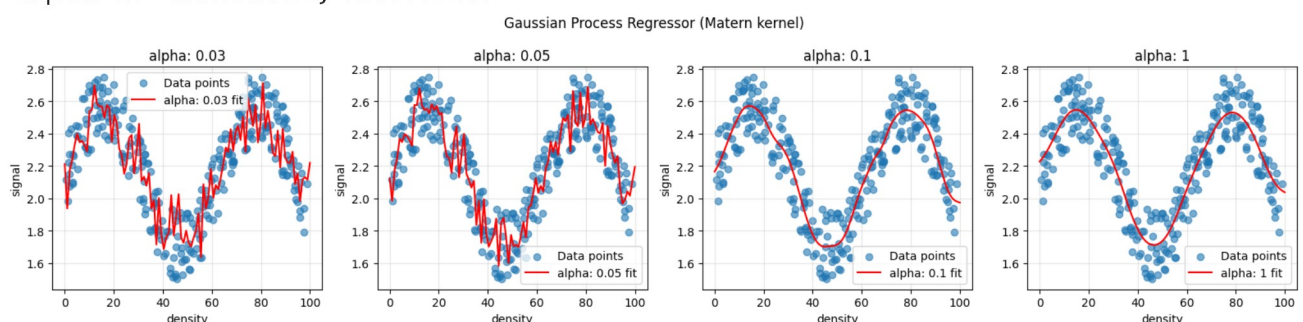
```
In [40]: run_n_test_gpr("Gaussian Process Regressor (RBF kernel)", RBF(length_scale=2, length_scale=
```

```
alpha: 0.03: MAE:0.1955, MSE:0.0649
alpha: 0.05: MAE:0.1225, MSE:0.0194
alpha: 0.1:  MAE:0.1234, MSE:0.0193
alpha: 1:   MAE:0.1293, MSE:0.0213
```



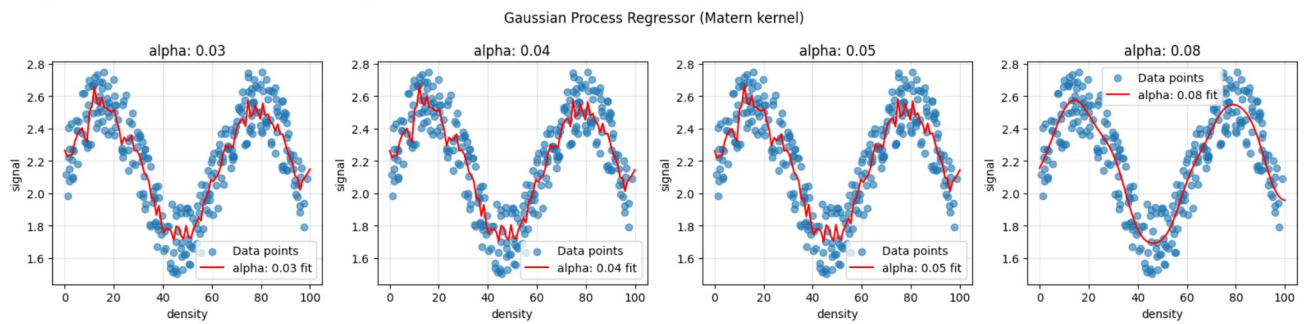
```
In [41]: run_n_test_gpr("Gaussian Process Regressor (Matern kernel)", Matern(length_scale=0.1, length_scale=
```

```
alpha: 0.03: MAE:0.1364, MSE:0.0287
alpha: 0.05: MAE:0.1300, MSE:0.0263
alpha: 0.1:  MAE:0.1234, MSE:0.0197
alpha: 1:   MAE:0.1282, MSE:0.0211
```



```
In [42]: run_n_test_gpr("Gaussian Process Regressor (Matern kernel)", RationalQuadratic(length_scale=
```


alpha: 0.03: MAE:0.1299, MSE:0.0227
 alpha: 0.04: MAE:0.1298, MSE:0.0225
 alpha: 0.05: MAE:0.1295, MSE:0.0224
 alpha: 0.08: MAE:0.1226, MSE:0.0193



Expected Graph Behavior

- **Shape:** Extremely smooth curves with uncertainty bands
- **Visual Result:** Beautiful smooth fit with confidence intervals showing prediction uncertainty
- **Uncertainty:** Higher uncertainty in regions with sparse data
- **Kernel Dependent:** Behavior heavily depends on kernel choice and hyperparameters
- **Interpolation:** Passes exactly through training points (if no noise)

When to Use

- **Ideal Scenarios:** Small-to-medium datasets, uncertainty quantification needed, smooth relationships, active learning
- **Adequate Scenarios:** Expensive experiments, Bayesian optimization, scientific modeling
- **Avoid When:** Large datasets ($O(n^3)$ complexity), discrete/categorical relationships, real-time applications
- **Industry Applications:** Materials science, drug discovery, environmental modeling, hyperparameter optimization

Neural Network (MLP)

Inspired by biological neurons, modern deep learning roots in 1940s-1980s

- **Core Philosophy:** Universal function approximation through layered non-linear transformations
- **Mathematical Foundation:** Composition of linear transformations with non-linear activations
- **Practical Motivation:** Can theoretically approximate any continuous function

Assumptions & Characteristics

- **Core Assumption:** Complex relationships can be learned through hierarchical feature learning
- **Mathematical Form:** $f(x) = W_L \sigma(W_{L-1} \sigma(\dots \sigma(W_1 x + b_1) \dots))$
- **Flexibility:** Extremely high with sufficient neurons and layers
- **Learning:** Gradient-based optimization (backpropagation)

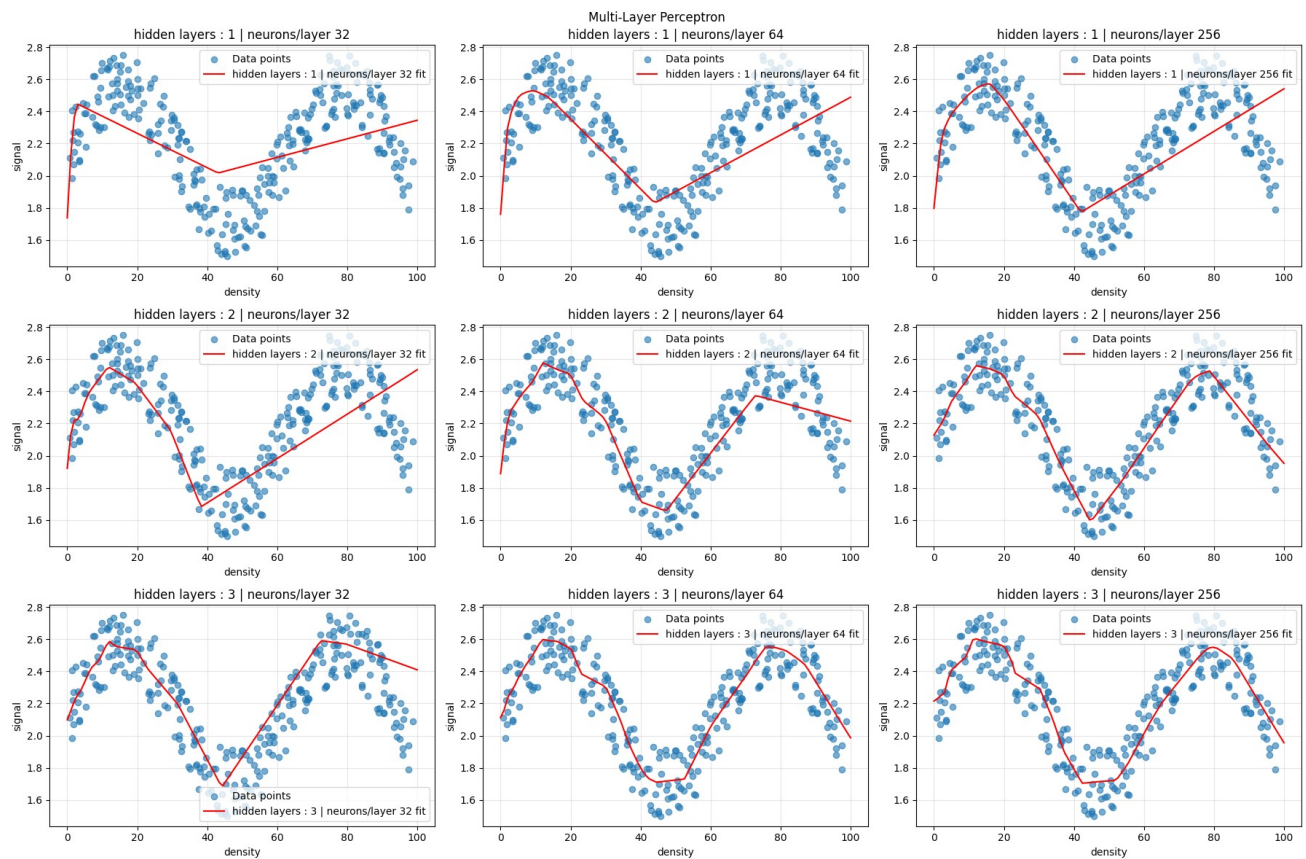
```
In [46]: import tensorflow as tf
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.models import Sequential
```

Key Hyperparameters

- **Hidden_layer_sizes:** Architecture specification (most critical)
 - Shallow (1 layer, 10-100 neurons): Simple non-linearities
 - Medium (1-2 layers, 50-500 neurons): Most regression problems
 - Deep (3+ layers): Complex patterns, risk of overfitting
- **Activation:** Non-linear activation function
 - ReLU: Default choice, computationally efficient
 - Tanh: Smooth, good for regression, slower convergence
 - Sigmoid: Historical choice, vanishing gradient problems
- **Alpha:** L2 regularization strength (0.0001-1.0)
- **Learning_rate:** Step size for gradient descent
 - Adaptive: adam, rmsprop (usually better)
 - Constant: sgd (requires careful tuning)
- **Max_iter:** Maximum iterations (100-10000+)
- **Early_stopping:** Prevent overfitting with validation monitoring

```
In [50]: run_n_test_mlp("Multi-Layer Perceptron", [1, 2, 3], [32, 64, 256])
```

```
2/2 ██████████ 0s 53ms/step
hidden layers : 1 | neurons/layer 32: MAE:0.2433, MSE:0.0773
4/4 ██████████ 0s 8ms/step
2/2 ██████████ 0s 53ms/step
hidden layers : 1 | neurons/layer 64: MAE:0.2210, MSE:0.0679
4/4 ██████████ 0s 9ms/step
2/2 ██████████ 0s 56ms/step
hidden layers : 1 | neurons/layer 256: MAE:0.2121, MSE:0.0648
4/4 ██████████ 0s 8ms/step
2/2 ██████████ 0s 63ms/step
hidden layers : 2 | neurons/layer 32: MAE:0.2106, MSE:0.0660
4/4 ██████████ 0s 7ms/step
2/2 ██████████ 0s 64ms/step
hidden layers : 2 | neurons/layer 64: MAE:0.1594, MSE:0.0342
4/4 ██████████ 0s 8ms/step
2/2 ██████████ 0s 60ms/step
hidden layers : 2 | neurons/layer 256: MAE:0.1297, MSE:0.0234
4/4 ██████████ 0s 8ms/step
2/2 ██████████ 0s 75ms/step
hidden layers : 3 | neurons/layer 32: MAE:0.1621, MSE:0.0441
4/4 ██████████ 0s 8ms/step
2/2 ██████████ 0s 90ms/step
hidden layers : 3 | neurons/layer 64: MAE:0.1229, MSE:0.0202
4/4 ██████████ 0s 9ms/step
2/2 ██████████ 0s 72ms/step
hidden layers : 3 | neurons/layer 256: MAE:0.1270, MSE:0.0202
4/4 ██████████ 0s 7ms/step
```



Expected Graph Behavior

- **Shape:** Can produce extremely smooth continuous curves
- **Visual Result:** Very smooth, can approximate virtually any function shape
- **Overfitting Risk:** High without proper regularization - may fit noise
- **Training Sensitivity:** Results may vary significantly with different initializations
- **Boundary Behavior:** Generally well-behaved, smooth extrapolation

When to Use (General Guidelines)

- **Ideal Scenarios:** Complex non-linear relationships, large datasets, high accuracy requirements
- **Adequate Scenarios:** Medium-sized datasets, feature interactions important, smooth relationships
- **Avoid When:** Small datasets, interpretability crucial, simple relationships, limited computational resources
- **Industry Applications:** Computer vision, natural language processing, complex regression problems, function approximation

Key Insights

Bias-Variance Tradeoff

- **High Bias Models:** Linear regression will consistently underfit
- **High Variance Models:** High-degree polynomials and deep networks may overfit
- **Balanced Models:** SVR, Gaussian processes, and well-tuned neural networks often provide the best balance

Smoothness Considerations

- **Smooth Functions:** Gaussian Process, Neural networks, SVR (RBF), polynomial regression
- **Non-smooth Functions:** Decision trees, random forests (somewhat)
- **Piecewise Smooth:** KNN

Practical Recommendations

- Start with **polynomial regression (degree 4-6)** as a baseline
- Try **SVR with RBF kernel** for robust smooth fits
- Use **Gaussian processes** when you need uncertainty quantification
- Consider **gradient boosting** for a good balance of performance and interpretability
- Try **neural networks** for complex patterns with sufficient data
- Avoid **linear models** and **single decision trees** for data with complex patterns

Tree-Based Model Hierarchy for Sinusoidal Data

- **Gradient Boosting > Random Forest > Single Decision Tree**
- Boosting methods generally produce smoother approximations due to sequential error correction
- Random forests provide some smoothing through averaging but retain step-like characteristics
- Single trees create the most jagged, discontinuous fits

Hyperparameter Sensitivity

- **Most Sensitive:** Neural networks, SVM (gamma, C parameters)
 - **Moderately Sensitive:** Polynomial degree, KNN (k value), Random Forest (n_estimators, max_depth)
 - **Least Sensitive:** Linear regression (no hyperparameters to tune)
-

Model Selection Framework

Performance Hierarchy of model (Dataset-Dependent)

- For **smooth, continuous relationships** (like sinusoidal data):
 1. **Gaussian Process** (with appropriate kernel) - Optimal for small-medium datasets
 2. **Neural Networks** (with proper regularization) - Best for large datasets
 3. **SVR with RBF kernel** - Excellent middle ground
 4. **Polynomial Regression** (degree 4-8) - Simple and effective
- For **complex, non-smooth relationships**:
 1. **Gradient Boosting** (XGBoost, LightGBM) - Best for tabular data
 2. **Random Forest** - Robust general-purpose choice
 3. **Neural Networks** - With sufficient data and tuning
- For **simple relationships**:
 1. **Linear Regression** - When truly linear

2. **Polynomial Regression** (low degree) - For simple curves
3. **Ridge/Lasso Regression** - With many features

Computational Complexity Considerations

- **Training Time:** Linear < Polynomial < Tree-based < SVM < Neural Networks < GP
- **Prediction Time:** Linear $\approx$ Polynomial < Tree-based < SVM < Neural Networks < GP
- **Memory Usage:** Linear < Polynomial < Tree-based < Neural Networks < SVM < GP
- **Scalability:** Tree-based methods and Neural Networks scale best to large datasets

Bias-Variance-Interpretability Triangle

- **Low Bias, Low Variance:** GP, well-tuned Neural Networks
- **Low Bias, High Variance:** High-degree polynomials, overfitted trees
- **High Bias, Low Variance:** Linear regression, shallow trees
- **High Interpretability:** Linear regression, Decision trees, Low-degree polynomials
- **Low Interpretability:** Neural networks, SVMs, GP, Ensemble methods

Practical Model Selection Strategy

1. **Start Simple:** Begin with linear regression and low-degree polynomial
2. **Add Complexity Gradually:** Try SVR with RBF, then tree-based methods
3. **Consider Data Size:**
 - Small (<1000): GP, SVR, polynomial regression
 - Medium (1# Comprehensive ML Models Analysis: Understanding Model Behavior and Characteristics